

Appunti di Algoritmi per l'ingegneria

Geppino Pucci - UNIPD

FILIPPO GARGIULO*

Febbraio - Giugno 2026

Indice

1	Introduzione e definizioni di base	2
1.1	Definizione di algoritmo	2
1.2	Correttezza e complessità	3
2	Divide and Conquer	5
2.1	Albero delle chiamate	5
2.2	Trova il massimo	6
2.3	Ricorrenze	7
2.3.1	Unfolding e induzione parametrica	8
2.3.2	Albero delle ricorrenze	8
2.3.3	Il Master's theorem	10
2.4	Operazioni tra matrici	15
2.4.1	Strassen	15
2.4.2	Matrici Supercircolanti	18
2.5	Polinomi e FFT	20
2.5.1	Operazioni nelle rappresentazioni	20
2.5.2	Conversione Naive	21
2.5.3	Discrete Fourier Transform	23
2.5.4	Proprietà di Sottostruttura e FFT	25
2.5.5	Esempi notevoli	27
3	Dynamic Programming	30
3.1	Memoizzazione	30
3.2	Paradigma generale DP	31
3.3	Problemi su stringhe e LCS	32
3.3.1	Sottostruttura	33
3.4	Matrix Chain Multiplication	35
4	Greedy	38
4.1	Task Scheduling	38
4.2	Compressione dati	40

*Mail per segnalare errori: filippo.gargiulo@studenti.unipd.it

§1 Introduzione e definizioni di base

Esempio di codice su $\text{\LaTeX}$:

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6
7      //codice di esempio
8
9      return 0;
10 }
11

```

§1.1 Definizione di algoritmo

Definizione 1.1 (Algoritmo). Un *algoritmo* è una sequenza di passi finita e deterministica¹. I passi elementari dell'algoritmo si dicono *istruzioni*, e l'insieme di tutte le possibili istruzioni che si possono dare ad un algoritmo costituisce un *modello computazionale*. Un algoritmo ha un *input* e un *output*, ed è quindi sempre anche una funzione

$$A : I \rightarrow O$$

dove I è detto insieme di input, e O insieme di output.

Definizione 1.2 (Problema Computazionale). Un problema computazionale Π è un sottoinsieme dell'insieme

$$\Pi \subseteq \mathcal{I} \times \mathcal{S}$$

Tale che per ogni elemento $i \in \mathcal{I}$, Π contenga almeno un elemento (i, s) per qualche $s \in \mathcal{S}$. L'insieme $\mathcal{I}$ si dice insieme delle *istanze* del problema, e $\mathcal{S}$ si dice insieme delle soluzioni.

Definizione 1.3 (Algoritmo che risolve un problema). Si dice che un algoritmo $A_\pi : I \rightarrow O$ risolve un problema computazionale Π se $\mathcal{I} = I$ e $\mathcal{S} = O$, e inoltre abbiamo che per ogni $i \in \mathcal{I}$:

$$A_\pi(i) = s \implies i \Pi s$$

Esempio 1.4 (Raggiungibilità in grafi diretti)

Definiamo il seguente problema computazionale:

Input: Un grafo diretto $G = (V, E)$ e due nodi v_1 e v_2 .

Output: Un cammino su G che parte da v_1 e arriva a v_2 , oppure un output ϵ , che indica che un tale cammino non esiste (ovvero il cammino vuoto).

Introduciamo inoltre un po' di notazione

Definizione 1.5 (Stella di Kleene). Dato un insieme (di solito numerabile) A , chiamiamo A^* la sua *stella di Kleene*, ovvero l'insieme

$$A^* := A^{\mathbb{N}} \cup \bigcup_{n=0}^{\infty} A^n$$

¹Non vedremo algoritmi randomizzati in questo corso.

di tutte le sequenze, possibilmente finite o vuote (A^0 contiene la sola sequenza vuota) a valori in A . Denotiamo una sequenza con le parentesi angolate:

$$s = \langle a_1, a_2, \dots, a_k \rangle \in A^*$$

con $a_i \in A$.

§1.2 Correttezza e complessità

Definizione 1.6 (Correttezza di un algoritmo). Dimostrare che un algoritmo A_Π è corretto per un problema computazionale Π vuol dire dimostrare che il predicato

$$(A_\Pi(i) = s) \implies (i \Pi s)$$

è valido per ogni $i \in \mathcal{I}$.

e questo era facile, meno chiaro è il concetto di complessità:

Definizione 1.7 (Tempo di esecuzione di un algoritmo). Dato un algoritmo A_π e un'istanza i , il suo *tempo di esecuzione* è la somma dei tempi di esecuzione di ciascuna istruzione eseguita dall'algoritmo:

$$t_{A_\pi, i} = \sum_{\text{istruzioni } istr \text{ eseguite da } A_\pi(i)} c_{istr}$$

per qualche scelta dei costi $c_{istr} \in \mathbb{R}^+ \cup \{0\}$.

questo di base è la definizione più generale di modello di costo, ma da usare nella pratica sarebbe completamente impraticabile. Per fare un'*analisi asintotica* di solito associamo un costo 1 ad alcune operazioni, ed un costo 0 ad altre (in particolare, associamo un costo 1 alle operazioni che *dominano la complessità*).

Definizione 1.8 (Taglia). Data un'istanza $i \in \mathcal{I}$ per qualche algoritmo A_π definiamo la sua *taglia* $|i| \in \mathbb{N}$ un numero intero non negativo che rappresenta *ragionevolmente* la grandezza della codifica dell'istanza i .

Forti di questa definizione allora possiamo definire

Definizione 1.9 (Funzione di complessità). Dato un algoritmo A_π di insieme di istanze $\mathcal{I}$, possiamo quozientare $\mathcal{I}$ rispetto alla relazione $\sim$ di "avere la stessa taglia", e dunque otteniamo la *funzione di complessità* (in tempo) per l'algoritmo A_π :

$$T_{A_\pi} : \mathcal{I}/\sim \subseteq \mathbb{N} \rightarrow \mathbb{R} \cup \{0\}$$

$$[i] \mapsto t_{A_\pi, i}$$

che è ben definita se la taglia è ben definita. Alcuni casi specifici sono la *worst-case complexity*

$$T_{A_\pi}(n) := \sup_{\substack{i \in \mathcal{I} \\ |i|=n}} \{t_{A_\pi, i}\}$$

la *best-case complexity*

$$T_{A_\pi}(n) := \inf_{\substack{i \in \mathcal{I} \\ |i|=n}} \{t_{A_\pi, i}\}$$

e la *average-case complexity*, in cui il tempo in funzione dell'istanza è visto come una variabile aleatoria, e quindi si definisce

$$T_{A_\pi}(n) := \mathbb{E}[t_{A_\pi, i}]$$

In particolare di solito la taglia di un'istanza è *il numero di bit* necessari a rappresentarlo in memoria. Per esempio un algoritmo che ha una complessità che dipende dal valore massimo di un certo intero ha complessità esponenziale, infatti un intero di valore u ha bisogno di

$$|u| = \lfloor \log_2(u) \rfloor + 1$$

bit per essere rappresentato, quindi in funzione della taglia un tale algoritmo avrebbe complessità

$$T$$

$$T(|u|) = 2^{|u|} = u$$

che è esponenziale! In generale algoritmi di questo tipo, che sembrano polinomiali per taglie "scelte male" si dicono *pseudopolinomiali*.

§2 Divide and Conquer

Un algoritmo divide and conquer (D&C) si basa su due fasi:

Divide: Ovvero, una fase in cui il problema che vogliamo risolvere è diviso in problemi analoghi su istanze di taglia più piccola.

Conquer: La fase in cui le soluzioni ai problemi di taglia più piccola vengono combinate per ottenere una soluzione al problema originale.

La proprietà importante che ci permette di applicare un algoritmo divide and conquer è

Definizione 2.1 (Proprietà di sottostruttura). Un problema Π si dice avere una certa *proprietà di sottostruttura* se esiste una taglia minima n_0 tale che per tutte le istanze di taglia $\leq n_0$ il problema sia "immediatamente risolvibile" (queste vengono dette *istanze di base*), e due funzioni D e C tali che

$$\begin{aligned} D : \mathcal{I} &\longrightarrow \mathcal{I}^* \\ i &\longmapsto \langle i_1, i_2, \dots, i_k \rangle \\ C : \mathcal{S}^* &\longrightarrow \mathcal{S} \\ \langle s_1, s_2, \dots, s_k \rangle &\longmapsto s \end{aligned}$$

tali che $|i_k| < |i|$ per ogni $i_k \in D(i)$, e inoltre se abbiamo

$$i_1 \Pi s_1, i_2 \Pi s_2, \dots, i_k \Pi s_k$$

abbiamo

$$i \Pi C(\langle s_1, s_2, \dots, s_k \rangle)$$

cioè possiamo fare sempre la fase di divide e la fase di conquer.

Insomma un algoritmo divide and conquer è costituito da quattro routine:

- [B]** Il caso base, in cui le soluzioni di taglia $\leq n_0$ vengono risolte direttamente, di solito tramite bruteforce.
- [D]** Viene applicata la routine $A_D(i)$ che applica il divide. Questa è la fase "top-down".
- [R]** Il programma ricorre su se stesso, ottenendo "gratuitamente" le soluzioni ai sottoproblemi più piccoli.
- [C]** Viene applicata la routine $A_C(\langle s_1, s_2, \dots, s_k \rangle)$ che applica la fase di conquer sulle soluzioni ottenute al passo precedente. Questa è la fase "bottom-up".

In generale la correttezza di un algoritmo divide and conquer si dimostra per induzione.

§2.1 Albero delle chiamate

La fase di divide di un algoritmo divide and conquer definisce un albero, il cosiddetto **albero delle chiamate**. In particolare il nodo i avrà come figli esattamente i nodi $D(i)$, e saranno foglie solo i nodi di taglia minore o uguale di n_0 .

Questo è soltanto concettuale, in effetti in macchina lo stack delle chiamate ricorsive è un DFS-order di questo albero, che non viene mai esplicitamente costruito. In effetti non abbiamo mai neanche tutto l'albero in memoria, al più avremo un numero di chiamate asintotico all'altezza dell'albero, perchè l'algoritmo ricorre finchè non raggiunge una

foglia.

Il grosso problema del paradigma divide and conquer è che funziona in modo efficiente solo quando tutti i nodi sono assunti distinti, infatti ogni volta che la procedura di conquer viene attuata sulle soluzioni $\langle s_1, s_2, \dots, s_k \rangle$ tutte le soluzioni alle sottoistanze $s_1, \dots, s_k$ vengono perse, e quindi se dovessero essere di nuovo necessarie per risolvere un'altra sottoistanza andranno calcolate di nuovo. Questo problema, risolto dalla DP, si chiama **memorylessness**.

§2.2 Trova il massimo

Un primo esempio molto stupido di algoritmo divide and conquer è quello che risolve:

Problema Computazionale 2.2 (Trova il massimo)

Definiamo il seguente problema computazionale:

Input: Un elemento v di $\mathbb{R}^*$.

Output: Il massimo elemento di v .

Questo è risolvibile con un algoritmo molto semplice divide and conquer:

```
1  int trova_il_massimo(vector<int> &v, int l, int r){
2      if(v.size()==1) return v[0];
3      //caso base
4      int a1 = trova_il_massimo(v, ((l+r)/2)+1, r);
5      //ricorsione (e divide)
6      int a2 = trova_il_massimo(v, l, (l+r)/2);
7      //ricorsione (e divide)
8      return max(a1, a2);
9      //conquer
10 }
11
```

La cui correttezza si dimostra molto facilmente per induzione.

§2.3 Ricorrenze

Il problema degli algoritmi divide and conquer di solito è che sono **ricorsivi**, cioè di solito la loro funzione di complessità soddisfa una ricorrenza. Per fare un'analisi della complessità dell'algoritmo per trovare il max consideriamo la taglia di un'istanza la lunghezza del vettore (insomma il valore $n := r - l$), la complessità $T(n)$ soddisfa allora

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ T(\lceil \frac{n}{2} \rceil) + T(\lfloor \frac{n}{2} \rfloor) + 1 & \text{altrimenti.} \end{cases}$$

(qui associamo un running time 1 ai confronti tra elementi del vettore² e un running time di 0 a tutto il resto). In generale per un algoritmo generico bisogna considerare tre costi, T_{AD} il costo del divide, T_{AR} il costo del recurse e T_{AC} il costo del conquer. Generalmente T_{AR} sarà in funzione di $T(n)$ e gli altri due saranno in funzione solo di n . Generalmente quindi abbiamo

$$T(n) = \begin{cases} f(n) & \text{se } n < n_0 \\ T_{AD} + T_{AR} + T_{AC} & \text{altrimenti.} \end{cases}$$

nel caso del max abbiamo in effetti:

$$\begin{aligned} T_{AD} &= 0 \\ T_{AR} &= T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \\ T_{AC} &= 1. \end{aligned}$$

Abbiamo quindi delle informazioni sulla funzione di complessità, dobbiamo trovare il modo per ricavarne fuori qualcosa sull'andamento asintotico di T (possibilmente una formula chiusa, ma questo non è sempre possibile).

Definizione 2.3 (Big-O, Big-Omega, Big-Theta). Diciamo che

$$T(n) = \mathcal{O}(f(n))$$

se e solo se esiste una costante c e un n_0 tali che per ogni $n \geq n_0$ abbiamo

$$cf(n) \leq T(n)$$

viceversa diciamo che

$$T(n) = \Omega(f(n))$$

se e solo se esiste una costante c e un n_0 tali che per ogni $n \geq n_0$ abbiamo

$$cf(n) \geq T(n)$$

infine diciamo che

$$T(n) = \Theta(f(n))$$

se e solo se abbiamo contemporaneamente

$$T(n) = \mathcal{O}(f(n))$$

$$T(n) = \Omega(f(n)).$$

²NON AI CONFRONTI TRA INDICI!

§2.3.1 Unfolding e induzione parametrica

Una prima strategia per trovare un andamento asintotico è cercare di "unfoldare" uno step alla volta la ricorrenza. In questo modo spesso possiamo trovare un **guess** sull'andamento asintotico (o possiamo dimostrare una formula chiusa per un sottoinsieme di $\mathbb{N}$ particolare, per esempio nel caso del max per le potenze di 2). Una volta ottenuto il guess, se non possiamo trovare direttamente una formula chiusa (dimostrandola per induzione), possiamo accontentarci di dimostrare le disuguaglianze

$$T(n) \geq c_1 f(n)$$

$$T(n) \leq c_2 f(n)$$

in modo da dimostrare che $T(n) = \Theta(f(n))$. A questo fine torna utile la tecnica dell'**induzione parametrica**, cioè cercare di dimostrare disuguaglianze per induzione lasciando parametri liberi di variare nelle funzioni con cui stiamo boundando il tempo. Per esempio per il max dimostriamo per induzione le disuguaglianze

$$T(n) \leq an + b$$

$$T(n) \geq cn + d$$

che ci permette di dire che $T(n) = \Theta(n)$.

§2.3.2 Albero delle ricorrenze

Un'altra possibilità consiste nello "svolgere completamente la ricorrenza" prendendo un'istanza worst-case e costruendo il suo albero delle chiamate. A questo punto a ogni nodo associamo una quantità che chiamiamo *work*, ovvero il tempo $T_D + T_C$ per i nodi interni e T_B per le foglie. In questo modo sommando il work su tutti i nodi otteniamo esattamente $T(n)$. Questo di solito per algoritmi di divide and conquer con tempi di divide e di conquer più o meno costanti è la strategia più efficace, anche perchè permette di visualizzare completamente quello che stiamo facendo. Per esempio nel caso del max abbiamo che i nodi interni contribuiscono esattamente con work uguale a

$$T_D + T_C = 0 + 1$$

mentre i nodi foglia hanno costo 0. Allora dato che, se abbiamo un array di n elementi, l'albero della ricorrenza ha esattamente $n - 1$ nodi interni e n nodi foglia, abbiamo che $T(n)$ è esattamente

$$T(n) = (n - 1)1 + n0 = n - 1$$

per un'istanza worst case. Un'albero

Definizione 2.4 (funzioni s, f, w). Dato un albero delle chiamate definiamo la funzione $s : \mathbb{N} \rightarrow \mathbb{N}$ il numero di sottoistanze $s(n)$ generate da un'istanza di taglia n . D'altra parte definiamo la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ la funzione che associa alla taglia n di un'istanza la taglia delle sottoistanze generate $f(n)$. Infine definiamo la funzione *work* $w : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ come la somma dei tempi $T_D + T_C$ per i nodi interni e T_B per i nodi foglia. In altre parole se le funzioni s, f, w sono ben definite (non è detto lo siano, succede solo per specifiche ricorrenze) la ricorrenza assume la forma:

$$T(n) = \begin{cases} T_0 & \text{se } n \leq n_0 \\ s(n)T(f(n)) + w(n) & \text{altrimenti.} \end{cases}$$

notiamo che per come abbiamo definito il divide and conquer f è una contrazione, cioè $f(n) < n$ per ogni n .

Poter definire queste funzioni restringe la generalità del nostro ragionamento, ma in ogni caso ci permette di trovare la soluzione a una grande quantità di equazioni di ricorrenza.

Definizione 2.5 (Composizione). Denotiamo con $f^k(n)$ la funzione f composta con se stessa k volte.

Esempio 2.6

La funzione

$$f(n) = \left\lceil \frac{n}{2} \right\rceil$$

composta con se stessa k volte ha formula chiusa:

$$f^k(n) = \left\lceil \frac{\left\lceil \frac{\dots}{2} \right\rceil}{2} \right\rceil = \left\lceil \frac{n}{2^k} \right\rceil$$

Definizione 2.7 (Funzione ausiliaria). Definiamo la *funzione ausiliaria* per un albero delle ricorrenze come la funzione

$$f^*(n, n_0) = \max \left\{ k \in \mathbb{N} : f^k(n) > n_0 \right\}$$

cioè è la profondità dell'albero oltre la quale tutte le istanze hanno taglia $\leq n_0$. Se $n < n_0$ definiamo $f^*(n, n_0) = -1$.

Notiamo che calcolare questa funzione di solito è molto facile:

Esempio 2.8

Se $f(n) = \left\lceil \frac{n}{2} \right\rceil$ abbiamo che il k che stiamo cercando per calcolare la funzione ausiliaria è

$$f^k(n) > n_0 \iff \frac{n}{2^k} > \left\lceil \frac{n}{2^k} \right\rceil > n_0$$

quindi k soddisfa

$$k < \log_2 n - \log_2 n_0$$

con questi ingredienti possiamo derivare una formula chiusa:

Teorema 2.9 (Formula esplicita di una ricorrenza su un albero)

Una ricorrenza $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ della forma

$$T(n) = \begin{cases} T_0 & \text{se } n < n_0 \\ s(n)T(f(n)) + w(n) & \text{altrimenti.} \end{cases}$$

con le funzioni s, f, w definite prima, ha formula esplicita

$$T(n) = \sum_{l=0}^{f^*(n, n_0)} \left(\left[\prod_{j=0}^{l-1} s(f^j(n)) \right] w(f^l(n)) \right) + T_0 \prod_{j=0}^{f^*(n, n_0)} s(f^j(n)).$$

Dimostrazione. L'idea è sommare livello per livello sull'albero delle ricorrenze. Per le assunzioni che abbiamo fatto per poter scrivere le funzioni s, f, w ci basta sommare il work del livello (che è uguale per tutti i nodi) tante volte quanti sono i nodi, cioè se l è il livello (la radice è al livello 0 e le foglie al livello m) abbiamo che

$$\# \text{ nodi al livello } l = s(f^0(n))s(f^1(n))s(f^2(n)) \dots s(f^{l-1}(n)) = \prod_{j=0}^{l-1} s(f^j(n))$$

d'altra parte il lavoro al livello l è semplicemente $w(f^l(n))$ e quindi abbiamo che

$$\sum_{l=0}^{f^*(n, n_0)} \left(\left[\prod_{j=0}^{l-1} s(f^j(n)) \right] w(f^l(n)) \right)$$

a questo dobbiamo aggiungere il work delle foglie, che è semplicemente il numero di nodi del livello $f^*(n, n_0)$ (le foglie) per il costo di una foglia T_0 .

$$T_0 \prod_{j=0}^{f^*(n, n_0)} s(f^j(n))$$

e sommando queste due quantità otteniamo la formula della tesi. $\square$

§2.3.3 Il Master's theorem

Il teorema definitivo che ci permette di calcolare direttamente l'andamento asintotico a partire dalla ricorrenza è il cosiddetto Master's Theorem. La prima versione, più debole, si occupa delle ricorrenze della forma

$$T(n) = \begin{cases} T_0 & \text{se } n = 1 \\ aT\left(\frac{n}{b}\right) + w(n) & \text{altrimenti.} \end{cases}$$

con $a \geq 1$ e $b \geq 2$ interi. In effetti qui stiamo assumendo che n sia una potenza di b , ma in ogni caso l'andamento asintotico vedremo che non cambierà.

Definizione 2.10 (Funzione di soglia). Data una ricorrenza $T(n)$ della forma sopra, la sua *funzione di soglia* è definita come $n^{\log_b a}$.

Teorema 2.11 (Master's theorem)

Data una ricorrenza $T(n)$ della forma

$$T(n) = \begin{cases} T_0 & \text{se } n = 1 \\ aT\left(\frac{n}{b}\right) + w(n) & \text{altrimenti.} \end{cases}$$

abbiamo tre casi, in base a come si comporta asintoticamente la funzione di soglia con la funzione di lavoro:

Il lavoro è tutto sulle foglie Se esiste un $\varepsilon > 0$ per cui la funzione di lavoro

$$w(n) = \mathcal{O}\left(n^{\log_b a - \varepsilon}\right)$$

allora abbiamo $T(n) = \mathcal{O}\left(n^{\log_b a}\right)$, in particolare se $T_0 > 0$ abbiamo anche:

$$T(n) = \Theta\left(n^{\log_b a}\right).$$

Il lavoro è bilanciato Se la funzione di lavoro soddisfa:

$$w(n) = \Theta\left(n^{\log_b a}\right)$$

allora abbiamo

$$T(n) = \Theta\left(n^{\log_b a} \log n\right)$$

Il lavoro è tutto sui nodi interni Infine se abbiamo che esiste un $\varepsilon > 0$ tale che

$$w(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$$

e inoltre la funzione di lavoro è *regolare*, cioè esiste una costante $c \in (0, 1)$ tale che per ogni n è soddisfatta la seguente disuguaglianza

$$aw\left(\frac{n}{b}\right) \leq cw(n)$$

allora si ha

$$T(n) = \Theta(w(n)).$$

Dimostrazione. Applichiamo la formula esplicita di una ricorrenza su un albero imponendo $s(n) = a$ e $f(n) = n/b$:

$$T(n) = \sum_{l=0}^{\log_b n - 1} a^l w\left(\frac{n}{b^l}\right) + T_0 a^{\log_b n}$$

che d'altra parte diventa

$$T(n) = \sum_{l=0}^{\log_b n - 1} a^l w\left(\frac{n}{b^l}\right) + T_0 n^{\log_b a}$$

da cui vediamo da dove spunta il termine della funzione soglia. Questa equazione ci dice già da subito che

$$T(n) = \Omega\left(n^{\log_b a}\right)$$

Ora vediamo i tre casi:

Il lavoro è tutto sulle foglie Se il work è asintoticamente minore della funzione soglia significa che esiste una costante c tale che

$$w(n) \leq cn^{\log_b a - \varepsilon}$$

per ogni n . Sostituendo:

$$\sum_{l=0}^{\log_b n - 1} a^l w\left(\frac{n}{b^l}\right) + T_0 n^{\log_b a} \leq \sum_{l=0}^{\log_b n - 1} a^l c \left(\frac{n}{b^l}\right)^{\log_b a - \varepsilon} + T_0 n^{\log_b a}$$

che portando fuori tutto dalla sommatoria:

$$\dots = cn^{\log_b a - \varepsilon} \sum_{l=0}^{\log_b n - 1} \left(\frac{a}{b^{\log_b a - \varepsilon}}\right)^l + T_0 n^{\log_b a} = cn^{\log_b a - \varepsilon} \sum_{l=0}^{\log_b n - 1} (b^\varepsilon)^l + T_0 n^{\log_b a}$$

che è una serie geometrica, e quindi

$$\dots = cn^{\log_b a - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1} + T_0 n^{\log_b a}$$

da cui

$$\dots = cn^{\log_b a - \varepsilon} \frac{n^\varepsilon - 1}{b^\varepsilon - 1} + T_0 n^{\log_b a} \leq cn^{\log_b a - \varepsilon} n^\varepsilon \frac{1}{b^\varepsilon - 1} + T_0 n^{\log_b a} = \frac{c}{1 - b^\varepsilon} n^{\log_b a} + T_0 n^{\log_b a}$$

che implica proprio che $T(n) = \mathcal{O}\left(n^{\log_b a}\right)$, che implica la tesi.

Il lavoro è bilanciato Come prima stimiamo il work sia dall'alto che dal basso con $c_1 n^{\log_b a}$ e $c_2 n^{\log_b a}$ per c_1, c_2 opportunamente scelte:

$$c_1 n^{\log_b a} \frac{1}{a^l} = c_1 \left(\frac{n}{b^l}\right)^{\log_b a} \leq w\left(\frac{n}{b^l}\right) \leq c_2 \left(\frac{n}{b^l}\right)^{\log_b a} = c_2 n^{\log_b a} \frac{1}{a^l}$$

che sostituendo nella sommatoria ci danno una stima dall'alto e dal basso del tempo

$$c_1 n^{\log_b a} \sum_{l=0}^{\log_b n - 1} \frac{a^l}{a^l} + T_0 n^{\log_b a} \leq T(n) \leq c_2 n^{\log_b a} \sum_{l=0}^{\log_b n - 1} \frac{a^l}{a^l} + T_0 n^{\log_b a}$$

cioè semplificando le sommatorie

$$c_1 n^{\log_b a} \log_b n + T_0 n^{\log_b a} \leq T(n) \leq c_2 n^{\log_b a} \log_b n + T_0 n^{\log_b a}$$

che ci dà proprio $T(n) = \Theta\left(n^{\log_b a} \log n\right)$ come volevamo.

Il lavoro è tutto sui nodi interni Si dimostra facilmente per induzione dalla condizione di regolarità che

$$a^l w\left(\frac{n}{b^l}\right) \leq c^l w(n)$$

d'altra parte abbiamo per l'asintotico che esiste una costante c_1 tale che

$$w(n) \geq c_1 n^{\log_b a + \varepsilon} \geq c_1 n^{\log_b a}$$

ora, per il bound dal basso ci basta notare che possiamo stimare la sommatoria con il termine per $l = 0$.

$$T(n) = \sum_{l=0}^{\log_b n - 1} a^l w(n/b^l) + T_0 n^{\log_b a} \geq w(n) + T_0 n^{\log_b a} \geq w(n)$$

cioè

$$T(n) = \Omega(w(n)).$$

D'altra parte per il bound dall'alto possiamo usare le due disugugaglianze enunciate prima

$$T(n) \leq \sum_{l=0}^{\log_b n - 1} c^l w(n) + T_0 c_1 w(n) = T_0 c_1 w(n) + w(n) \sum_{l=0}^{\infty} c^l$$

e dato che $0 < c < 1$ quella serie geometrica converge, e quindi abbiamo proprio

$$T(n) \leq \dots \leq T_0 c_1 w(n) + w(n) \frac{1}{1-c} = \mathcal{O}(w(n))$$

che termina la dimostrazione.

Dunque abbiamo esaurito i casi, e il teorema è dimostrato. $\square$

Notiamo prima di andare avanti alcune cose: la prima è che il terzo caso del teorema è sovraspecificato, infatti abbiamo che

Lemma 2.12 (Regolarità implica boundato dal basso)

Sia $w(n)$ una generica funzione di work regolare, cioè tale che esistono costanti a, b, c tali che

$$aw(n/b) \leq cw(n)$$

allora per qualche $\varepsilon > 0$ vale:

$$w(n) = \Omega(n^{\log_b a + \varepsilon})$$

Dimostrazione. Abbiamo già detto che si può dimostrare agilmente per induzione che

$$a^l w(n/b^l) \leq c^l w(n)$$

per ogni l intero. In particolare se imponiamo $l = \lfloor \log_b n \rfloor$ abbiamo

$$a^k a^{\log_b n} w(1/b^k) \leq c^{\log_b n} w(n)$$

dove $k = \log_b n - \lfloor \log_b n \rfloor$ e quindi abbiamo proprio

$$a^k n^{\log_b a} w(1/b^k) \leq n^{\log_b c} w(n)$$

che portando tutto da un lato e notando che essendo $c \in (0, 1)$ abbiamo $\log_b c < 0$ otteniamo la tesi

$$a^k w(1/b^k) n^{\log_b a - \log_b c} \leq w(n) \iff \Omega(n^{\log_b a - \log_b c}) = w(n)$$

come richiesto. $\square$

e la seconda cosa è che in effetti tutti i polinomi sono regolari

Lemma 2.13 (I polinomi sono regolari)

Se $w(n)$ è un polinomio allora per ogni $a, b \in \mathbb{N}$ esiste una costante $c \in (0, 1)$ tale che

$$aw(n/b) \leq cw(n)$$

Dimostrazione. Chiaramente la funzione

$$w(n) = n^k$$

è regolare, infatti basta scegliere $c = \frac{a}{b^k}$ per ottenere

$$a \frac{n^k}{b^k} = aw\left(\frac{n}{b}\right) \leq \frac{a}{b^k} w(n) = \frac{a}{b^k} n^k$$

che è ovviamente verificata. D'altra parte notiamo che se $x(n)$ e $y(n)$ sono regolari, anche le loro combinazioni lineari lo saranno, infatti se abbiamo che

$$ax(n/b) \leq c_1 x(n)$$

$$ay(n/b) \leq c_2 y(n)$$

abbiamo che la funzione $a_1 x(n) + a_2 y(n)$ sommando termine a termine le due disuguaglianze precedenti, è boundata da:

$$a_1 ax(n/b) + a_2 ay(n/b) \leq a_1 c_1 x(n) + a_2 c_2 y(n)$$

che scegliendo $c_3 = \max(c_1, c_2)$ è

$$a_1 ax(n/b) + a_2 ay(n/b) \leq a_1 c_1 x(n) + a_2 c_2 y(n) \leq c_3 (a_1 x(n) + a_2 y(n))$$

cioè è di nuovo regolare. Dunque abbiamo mostrato che le funzioni potenza sono regolari, e che combinazione lineare di funzioni regolari è regolare, e quindi abbiamo che tutti i polinomi sono regolari. $\square$

§2.4 Operazioni tra matrici

Definizione 2.14 (Taglia di un algoritmo sulle matrici). Dato un algoritmo che opera su matrici, definiamo la taglia di una matrice come il suo numero di righe. Quindi la taglia di un'istanza costituita da una sola matrice è n^2 .

Definizione 2.15 (Complessità di un algoritmo su matrici). Il modello di costo che adottiamo per gli algoritmi su matrici attribuisce costo unitario alle operazioni tra scalari, e costo nullo a tutte le altre operazioni.

Per sommare matrici tra loro possiamo usare l'algoritmo naïve, e questo è ottimale dato che ha una complessità uguale alla complessità richiesta a memorizzare le matrici di ingresso. Per moltiplicare le matrici la faccenda si fa più interessante, l'algoritmo naïve è:

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<vector<int>> matmul_naive(int N, vector<vector<int>> A,
6  vector<vector<int>> B){
7
8      vector<vector<int>> C(N,vector<int>(N,0));
9
10     for(int i = 0 ; i < N ; i++){
11         for(int j = 0 ; j < N ; j++){
12             // facciamo il prodotto scalare (come se fosse in
13             // notazione di Einstein)
14             for(int k = 0 ; k < N ; k++){
15                 C[i][j] += A[i][k] * B[k][j];
16             }
17         }
18     }
19
20     return C;

```

facendo un'analisi (trasformando ogni for in una sommatoria essenzialmente), otteniamo

$$T(n) = 2n^3 - n^2 = \Theta(n^3)$$

ci chiediamo allora se si possa fare di meglio.

§2.4.1 Strassen

Vogliamo applicare un algoritmo divide and conquer, la prima cosa che ci è utile è paddare la matrice, aggiungendo una cornice di 0 alle matrici A , B che vogliamo moltiplicare finchè non hanno le dimensioni di una potenza di 2:

$$\left(\begin{array}{c|c} A & 0 \\ \hline 0 & 0 \end{array} \right) \left(\begin{array}{c|c} B & 0 \\ \hline 0 & 0 \end{array} \right) = \left(\begin{array}{c|c} AB & 0 \\ \hline 0 & 0 \end{array} \right)$$

questo non intacca più di tanto la nostra complessità, in quanto aumenta il tempo solo in modo costante. L'idea dell'algoritmo è utilizzare la *proprietà frattale* del prodotto tra matrici, ovvero

$$\left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right] = \left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right]$$

dove i C_i sono

$$C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$$

per ogni i, j . Questo si può dimostrare facendo tanti conti con sommatorie, in modo brutto.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  template <typename T>
5
6  class matrix{
7      int dim;
8      vector<vector<T>> data;
9      vector<T> & operator [] (int i)
10     {
11         return data[i];
12     }
13     matrix<T> operator+(const matrix other) const
14     {
15
16     }
17
18     public:
19     matrix(int n) : dim(n), data(n, vector<T>(n, T())){ }
20     matrix(const vector<vector<T>> &V) : dim(V.size()), data(V){ }
21 }
22
23 matrix<int> matumlR (matrix<int> A, matrix<int> B){
24     return
25 }
26

```

l'idea di Strassen è simile a quella di Karatsuba, ed è di calcolare le somme

$$\begin{array}{ll}
 A_1 = A_{12} - A_{22} & B_1 = B_{21} + B_{22} \\
 A_2 = A_{11} + A_{22} & B_2 = B_{11} + B_{22} \\
 A_3 = A_{11} - A_{12} & B_3 = B_{11} + B_{12} \\
 A_4 = A_{11} + A_{21} & B_4 = B_{22} \\
 A_5 = A_{11} & B_5 = B_{12} - B_{22} \\
 A_6 = A_{22} & B_6 = B_{21} - B_{11} \\
 A_7 = A_{21} + A_{22} & B_7 = B_{11}
 \end{array}$$

da cui abbiamo che calcolando i prodotti

$$P_i = A_i B_i$$

per ogni i , i quadranti della matrice C sono

$$\begin{array}{l}
 C_1 = P_1 + P_2 - P_4 + P_6 \\
 C_2 = P_4 + P_5 \\
 C_3 = P_6 + P_7 \\
 C_4 = P_2 - P_3 + P_5 - P_7
 \end{array}$$

abbastanza crazy. La ricorsione associata a questo algoritmo dunque è

$$T(n) = 7T(n/2) + \frac{9}{2}n^2$$

che per il MT ha complessità $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$. Esplicitamente risolta con la formula chiusa in realtà otteniamo che

$$T(n) = 7n^{\log_2 7} - 6n^2$$

che ha una costante importante davanti. In effetti l'algoritmo di Strassen è davvero più efficiente solo per matrici parecchio grandi, cioè per matrici di dimensione paddata superiore di n , dove n è la più piccola potenza di 2 tale che

$$7n^{\log_2 7} - 6n^2 < 2n^3 - n^2$$

che è qualcosa come 1024, cioè si parla di matrici con più di un milione di elementi. Poi questo non tiene conto di fattori sperimentali, e quindi di solito Strassen è usato solo quando si lavora con oggetti molto molto grandi.

§2.4.2 Matrici Supercircolanti

Quando lavoriamo con classi di matrici specifiche possiamo sviluppare algoritmi più efficienti del semplice Strassen.

Definizione 2.16 (Matrice Supercircolante). Una matrice A quadrata $n \times n$ si dice *supercircolante di ordine n* se:

- $n = 1$, cioè è semplicemente uno scalare.
- Possiamo scrivere la matrice come

$$A = \left(\begin{array}{c|c} A_1 & A_2 \\ \hline A_2 & A_1 \end{array} \right)$$

dove A_1, A_2 sono a loro volta matrici supercircolanti di ordine $n/2$.

Chiaramente una matrice supercircolante ha sempre dimensioni una potenza di 2.

Esempio 2.17

Le due matrici

$$\left(\begin{array}{c|c} 1 & 2 \\ \hline 2 & 1 \end{array} \right), \left(\begin{array}{c|c} 3 & 4 \\ \hline 4 & 3 \end{array} \right)$$

sono supercircolanti. Allo stesso modo la matrice

$$\left(\begin{array}{c|c|c} 1 & 2 & 3 \\ \hline 2 & 1 & 4 \\ \hline 3 & 4 & 1 \\ \hline 4 & 3 & 2 \end{array} \right)$$

è supercircolante.

Lemma 2.18 (Condizione necessaria per essere supercircolante)

Per una matrice supercircolante, ogni riga è una permutazione delle altre.

Lemma 2.19 (Chiusura)

Siano A e B due matrici supercircolanti. Allora le matrici

$$AB$$

$$\lambda_1 A + \lambda_2 B$$

sono matrici supercircolanti.

Dimostrazione. Procediamo per induzione estesa. Il caso base è chiaro, supponiamo ora che tutte le matrici supercircolanti di ordine minore di n siano chiuse, allora se A e B sono supercircolanti di ordine n si ha

$$A = \left(\begin{array}{c|c} A_1 & A_2 \\ \hline A_2 & A_1 \end{array} \right)$$

$$B = \left(\begin{array}{c|c} B_1 & B_2 \\ \hline B_2 & B_1 \end{array} \right)$$

e la somma e il prodotto sono

$$AB = \left(\begin{array}{c|c} A_1 & A_2 \\ \hline A_2 & A_1 \end{array} \right) \left(\begin{array}{c|c} B_1 & B_2 \\ \hline B_2 & B_1 \end{array} \right) = \left(\begin{array}{c|c} A_1B_1 + A_2B_2 & A_1B_2 + A_2B_1 \\ \hline A_2B_1 + A_1B_2 & A_2B_2 + A_1B_1 \end{array} \right)$$

$$\lambda_1 A + \lambda_2 B = \left(\begin{array}{c|c} \lambda_1 A_1 + \lambda_2 B_1 & \lambda_1 A_2 + \lambda_2 B_2 \\ \hline \lambda_1 A_2 + \lambda_2 B_2 & \lambda_1 A_1 + \lambda_2 B_1 \end{array} \right)$$

che per l'ipotesi induttiva sono di nuovo supercircolanti. $\square$

Lemma 2.20 (Il prodotto di supercircolanti è commutativo)

Comunque date due matrici supercircolanti abbiamo

$$AB = BA$$

Il fatto che le matrici supercircolanti siano intrinsecamente ricorsive ci permette immediatamente di scrivere un algoritmo divide and conquer per la somma. In effetti un tale algoritmo ci permette di fare $\mathcal{O}(n^2)$ assegnazioni e $\mathcal{O}(n)$ somme, che non è sempre un vantaggio (anzi, spesso non lo è), ma still, è qualcosa. Dove la situazione si fa già più interessante è per il prodotto:

$$\begin{pmatrix} A_1 & A_2 \\ A_2 & A_1 \end{pmatrix} \begin{pmatrix} B_1 & B_2 \\ B_2 & B_1 \end{pmatrix} = \begin{pmatrix} A_1B_1 + A_2B_2 & A_1B_2 + A_2B_1 \\ A_2B_1 + A_1B_2 & A_2B_2 + A_1B_1 \end{pmatrix}$$

questi sono solo 4 prodotti distinti, e infatti brutalmente implementando questa cosa otteniamo un algoritmo che va come

$$T(n) = 4T(n/2) + n$$

(di nuovo senza contare le assegnazioni) che è subito $\Theta(n^2)$ per il MT. Ma possiamo fare di meglio! Usando un trucchetto alla strassen abbiamo:

$$U := (A_1 + A_2)(B_1 + B_2)$$

$$V := (A_1 - A_2)(B_2 - B_1)$$

che sviluppando e sommando ci danno

$$\frac{U + V}{2} = A_1B_1 + A_2B_2$$

$$\frac{U - V}{2} = A_2B_1 + A_1B_2$$

che ci permettono quindi di ricorrere con una ricorsione che è

$$T(n) = 2T(n/2) + 4n$$

che per il MT viene $\Theta(n \log n)$ (senza contare le assegnazioni)! Il codice è

```
1 #include <bits/stdc++.h>
2
```

§2.5 Polinomi e FFT

Definizione 2.21 (Rappresentazione). Dato un oggetto matematico generico p , una sua *rappresentazione* è un oggetto dal quale possiamo ricostruire in tutti i dettagli l'oggetto p (e viceversa). Se $\mathbf{a}$ è una rappresentazione di p scriveremo

$$\mathbf{a} \equiv p.$$

Sappiamo benissimo cos'è un polinomio a coefficienti complessi $p \in \mathbb{C}[x]$. A livello computazionale ci sono 2 modi importanti per rappresentare un polinomio:

Definizione 2.22 (Rappresentazione su n coefficienti). Il primo modo, importante, è coefficiente per coefficiente come numeri. In questo modo il polinomio diventa banalmente una lista di n coefficienti in $\mathbb{C}^*$. Un polinomio rappresentato come lista di n numeri si dice *rappresentato su n elementi*. Notiamo che questo implica che il grado del polinomio sia $\leq n - 1$.

Importante sarà pensare i polinomi come elementi di uno spazio vettoriale di dimensione il grado massimo con cui stiamo lavorando. Il *padding* di una rappresentazione consiste nell'aggiungere elementi neutri alla rappresentazione per uniformarne la dimensione a un certo standard (per esempio, se stiamo lavorando con polinomi di grado n torna comodo paddare tutti i polinomi di grado minore aggiungendo termini $0x^k$). Per il principio di identità dei polinomi abbiamo il seguente chiaro teorema:

Teorema 2.23 (Di interpolazione)

Dati n elementi (x_i, y_i) di $\mathbb{C} \times \mathbb{C}$ tali che $x_i \neq x_j$ per ogni coppia di indici i, j allora esiste uno e un solo polinomio di grado limitato da n che passa per tutti.

Questo teorema crea immediatamente un'altra rappresentazione di un polinomio

Definizione 2.24 (Rappresentazione per punti). Dato un vettore $\mathbf{x} \in \mathbb{C}^n$, un polinomio di grado p di grado limitato da n si dice *rappresentato per (n) punti in base $\mathbf{x}$* se lo scriviamo come

$$p \equiv (\mathbf{x}, \mathbf{y})$$

dove $\mathbf{y}$ è un elemento di $\mathbb{C}^n$ tale che $p(x_i) = y_i$ per ogni $0 \leq i < n$.

La rappresentazione per punti è *estendibile*, cioè possiamo passare da una rappresentazione ad un'altra facendo un padding. Purtroppo qui gli elementi che lasciano invariata la rappresentazione, se estendiamo il vettore $\mathbf{x}$ con un vettore $\mathbf{x}^E$ dovremo estendere il vettore $\mathbf{y}$ valutando $\mathbf{x}$ in p coordinata per coordinata.

Il punto di partenza allora per arrivare a FFT sarà quello di sviluppare **algoritmi di conversione efficienti** per le due rappresentazioni.

§2.5.1 Operazioni nelle rappresentazioni

Implementare operazioni di somma o differenza in rappresentazione per coefficienti è molto facile. La situazione già si complica per il prodotto, infatti abbiamo che, dati

$$p(x) = \sum_{j=0}^{n-1} a_j x^j$$

$$q(x) = \sum_{j=0}^{n-1} b_j x^j$$

il loro prodotto è

$$p(x)q(x) = \left(\sum_{j=0}^{n-1} a_j x^j \right) \left(\sum_{j=0}^{n-1} b_j x^j \right)$$

che espandendo viene

$$\dots = \sum_{j=0}^{2n-2} c_j x^j$$

ma notiamo che per ottenere un termine di grado j dobbiamo moltiplicare termini di grado k e grado h tali che $k + h = j$. Cioè abbiamo che

$$c_j = \sum_{k=0}^j a_k b_{j-k}$$

Questa formula è valida per tutti i j da 0 a $2n - 2$ se consideriamo di aver paddato inizialmente entrambi i vettori fino a $2n - 2$ in realtà il modo più intelligente è risparmiarsi le operazioni, riscrivendo la formula come

$$c_j = \sum_{k=\min\{0, j-n+1\}}^{\min\{j, n-1\}} a_k b_{j-k}$$

che è più brutto ma più efficiente: Stiamo arrivando da qualche parte:

Definizione 2.25 (Convoluzione). Dati due vettori $\vec{a}, \vec{b}$ definiamo l'operatore *convoluzione lineare* $\star : \mathbb{C}^n \times \mathbb{C}^n \rightarrow \mathbb{C}^{2n+1}$ tale che

$$(\vec{a} \star \vec{b})_j = \sum_{k=0}^j a_k b_{j-k}.$$

notiamo che scritta così la convoluzione è parecchio costosa, $\Theta(n^2)$ in particolare. Sommare polinomi in rappresentazione per punti è di nuovo banale. Sul prodotto invece moltiplicare le cose è estremamente più efficiente, se abbiamo due polinomi p, q di grado limitato da n rappresentati in base $\mathbf{x}$ con la lista $\mathbf{y}$ il prodotto potrà essere rappresentato come

$$p(x_i)q(x_i) = z_i = (p \cdot q)(x_i)$$

notiamo che tuttavia, perchè questa sia effettivamente una rappresentazione, abbiamo bisogno di allungare le liste in modo che la lista degli z_i sia lunga effettivamente $2n - 1$, e questo lo possiamo fare in $\Theta(n)$, tempo lineare! Questa è la chiave della FFT, **moltiplicare polinomi per punti è molto facile.**

§2.5.2 Conversione Naive

Vediamo ora come convertire da coefficienti a punti. Il modo naive dato z componente di $\mathbf{x}$ di moltiplicare ogni volta z^j per z , moltiplicarlo al coefficiente giusto e sommare tutto. Questo ci dà $2n - 1$ prodotti e $n - 1$ somme. Dato che a livello macchina in generale i prodotti sono meno efficienti delle somme per ridurre il numero di prodotti possiamo ricorrere alla cosiddetta regola di Horner:

$$\sum_{j=0}^{n-1} a_j x^j = a_0 + x(a_1 + x(a_2 + x(\dots)))$$

che ci permette di svolgere solo $n - 1$ prodotti ed $n - 1$ somme.

La conversione inversa si fa considerando che gli n punti ci danno un sistema lineare, nelle incognite i coefficienti del polinomio, tale che

$$\sum_{j=0}^{n-1} a_j x_i^j = p(x_i) = y_i$$

scritto in forma matriciale questo sistema è della forma

$$X\vec{a} = \vec{y}$$

dove X è la matrice dei coefficienti, che in questo caso è della forma

$$X = \begin{pmatrix} x_0^0 & x_0^1 & x_0^2 & \dots & x_0^{n-1} \\ x_1^0 & x_1^1 & x_1^2 & \dots & x_1^{n-1} \\ x_2^0 & x_2^1 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{n-1}^0 & x_{n-1}^1 & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{pmatrix}$$

che è una cosiddetta matrice di Vandermonde:

Definizione 2.26 (Matrice di Vandermonde). Dato un vettore $\vec{z} \in \mathbb{C}^n$ la sua *matrice di Vandermonde* è la matrice $n \times n$ tale che l'entrata a_{ij} è uguale a z_i^{j-1} .

Lemma 2.27 (Vandermonde)

Il determinante della matrice di Vandermonde V di un vettore $\mathbf{z} \in \mathbb{C}^n$ è sempre uguale a

$$\det V = \prod_{i < j} (z_i - z_j)$$

in particolare quindi una matrice di Vandermonde è invertibile se e solo se tutte le componenti del vettore $\mathbf{z}$ sono distinte.

Dimostrazione. Per induzione, vedi gli appunti di Geometria 1. □

Dunque quello che vorremmo fare è risolvere quel sistemone lineare, e farlo con Gauss purtroppo ci costa $\Theta(n^3)$. In realtà un modo più efficiente per fare ciò è con l'interpolazione di Lagrange:

Lemma 2.28 (Interpolazione di Lagrange)

Data una rappresentazione per punti $(\vec{x}, \vec{y})$ del polinomio p , abbiamo che, per ogni $x \in \mathbb{C}$ vale:

$$p(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k}^{n-1} (x - x_j)}{\prod_{j \neq k}^{n-1} (x_k - x_j)}$$

o analogamente, definito il polinomio

$$Q(x) := \prod_{j=0}^{n-1} (x - x_j)$$

possiamo esprimere la formula come:

$$Q_k(x) := \frac{1}{x - x_k} Q(x)$$

e dunque abbiamo che

$$p(x) = \sum_{k=0}^{n-1} y_k \frac{Q_k(x)}{Q(x_k)}.$$

Dimostrazione. Chiara. □

La cosa interessante di questa formula è che ci permette di passare dalla rappresentazione per punti a quella per coefficienti in $\Theta(n^2)$, infatti ci basta calcolare in tempo quadratico il polinomio $Q(x)$ (espandendo e applicando la proprietà distributiva), e poi calcolare ogni polinomio Q_k **utilizzando la divisione di Ruffini tra polinomi**, che ha tempo lineare. Infine sommando tutto otteniamo la rappresentazione per coefficienti.

Queste conversioni non ci permettono di moltiplicare più velocemente i polinomi purtroppo, perché sono entrambe in $\Theta(n^2)$.

§2.5.3 Discrete Fourier Transform

Definizione 2.29 (Base di Fourier). Dato un naturale n definiamo la sua *base di Fourier* come l'insieme

$$\Omega_n := \{\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}\}$$

dove ω è l' n -esima radice primitiva dell'unità, ovvero

$$\omega_n := e^{i \frac{2\pi}{n}}.$$

Definizione 2.30 (Trasformata discreta di Fourier). Dato un vettore $\vec{a}$, la sua *trasformata di Fourier* è il vettore $\vec{y} = DFT_n(\vec{a})$ avente componente

$$y_i = \sum_{j=0}^{n-1} a_j (\omega_n^i)^j = \sum_{j=0}^{n-1} a_j \omega_n^{ij}.$$

Ora il punto del nostro discorso è che fare la DFT del vettore dei coefficienti di un polinomio restituisce la rappresentazione per punti nella base di Fourier! Quindi se riuscissimo a calcolare efficientemente la DFT e la DFT inversa potremmo calcolare efficientemente anche la convoluzione di polinomi. La prima cosa che possiamo fare è

Lemma 2.31 (La trasformata di Fourier è lineare)

Comunque dati due vettori $\vec{a}, \vec{b}$ e scalari complessi λ_1, λ_2 abbiamo

$$DFT_n(\lambda_1 \vec{a} + \lambda_2 \vec{b}) = \lambda_1 DFT_n(\vec{a}) + \lambda_2 DFT_n(\vec{b}).$$

Dimostrazione. Segue dal fatto che tutte le operazioni che stiamo facendo durante la trasformata di Fourier sono lineari. $\square$

Dato che la DFT è un operatore lineare possiamo scrivere la sua matrice nella base canonica di $\mathbb{C}^n$, e quindi abbiamo:

Definizione 2.32 (Matrice di Fourier). La matrice nella base canonica dell'operatore DFT_n viene detta *matrice di Fourier*, ed è la matrice di Vandermonde della base di Fourier:

$$F_n := \begin{pmatrix} \omega_n^{0 \cdot 0} & \omega_n^{0 \cdot 1} & \omega_n^{0 \cdot 2} & \dots & \omega_n^{0 \cdot (n-1)} \\ \omega_n^{1 \cdot 0} & \omega_n^{1 \cdot 1} & \omega_n^{1 \cdot 2} & \dots & \omega_n^{1 \cdot (n-1)} \\ \omega_n^{2 \cdot 0} & \omega_n^{2 \cdot 1} & \omega_n^{2 \cdot 2} & \dots & \omega_n^{2 \cdot (n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \omega_n^{(n-1) \cdot 0} & \omega_n^{(n-1) \cdot 1} & \omega_n^{(n-1) \cdot 2} & \dots & \omega_n^{(n-1) \cdot (n-1)} \end{pmatrix}$$

dato che ω_n è una radice primitiva abbiamo che tutte le sue potenze sono distinte, cioè la base di Fourier è formata da vettori tutti distinti, il che implica che la DFT è sempre invertibile in quanto la matrice di Fourier ha sempre determinante diverso da 0. Notiamo che la matrice di Fourier è sempre **simmetrica**, in quanto gli esponenti commutano.

La prima cosa, importantissima, che notiamo è la seguente:

Lemma 2.33 (La trasformata di Fourier è conforme)

L'inversa della matrice di Fourier è

$$F_n^{-1} = \frac{1}{n} \begin{pmatrix} \omega_n^{-0 \cdot 0} & \omega_n^{-0 \cdot 1} & \omega_n^{-0 \cdot 2} & \dots & \omega_n^{-0 \cdot (n-1)} \\ \omega_n^{-1 \cdot 0} & \omega_n^{-1 \cdot 1} & \omega_n^{-1 \cdot 2} & \dots & \omega_n^{-1 \cdot (n-1)} \\ \omega_n^{-2 \cdot 0} & \omega_n^{-2 \cdot 1} & \omega_n^{-2 \cdot 2} & \dots & \omega_n^{-2 \cdot (n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \omega_n^{-(n-1) \cdot 0} & \omega_n^{-(n-1) \cdot 1} & \omega_n^{-(n-1) \cdot 2} & \dots & \omega_n^{-(n-1) \cdot (n-1)} \end{pmatrix}$$

ovvero abbiamo che, denotando con F_n^* l'aggiunta di F_n :

$$F_n^* F_n = n I_n$$

cioè la trasformata di Fourier è conforme con indice di conformità $\sqrt{n}$.

Dimostrazione. Una banale moltiplicazione, in notazione di Einstein abbiamo che

$$(F_n^* F_n)_{ij} = \omega_n^{-ik} \omega_n^{kj} = \sum_{k=0}^{n-1} \omega_n^{k(i-j)} = n \delta_{ij}$$

che è esattamente la matrice $n I_n$, come richiesto. $\square$

Questo si uniforma alle nostre conoscenze di Analisi complessa, che ci dicono che la Trasformata di Fourier normalizzata è un'isometria.

§2.5.4 Proprietà di Sottostruttura e FFT

Abbiamo quindi capito che fare la trasformata di Fourier o invertirla sono essenzialmente la stessa cosa. Come facciamo quindi a calcolare efficientemente la trasformata di Fourier di un vettore arbitrario? Prendiamo un vettore $\vec{a}$ e il suo polinomio associato $p(x)$ **restringiamoci al caso di n (il grado del polinomio) potenza di 2**, tenendo a mente il fatto che questo non ci fa troppo perdere di generalità grazie al padding. Ricorriamo alla proprietà di Sottostruttura generata dall'identità polinomiale

$$p^{[0]}(x^2) + xp^{[1]}(x^2) = p(x)$$

dove introduciamo la notazione $p^{[0]}$ e $p^{[1]}$ per denotare i vettori aventi rispettivamente i coefficienti

$$a_0, a_2, a_4, a_6, \dots$$

$$a_1, a_3, a_5, a_7, \dots$$

dove $a_1, a_2, a_3, \dots, a_n$ sono i coefficienti del polinomio $p(x)$. L'identità polinomiale per come è stata enunciata è ovvia, ma immediatamente valutando entrambi i lati dell'identità in ω_n^j otteniamo qualcosa di poco ovvio

$$p(\omega_n^j) = p^{[0]}(\omega_n^{2j}) + \omega_n^j p^{[1]}(\omega_n^{2j})$$

a sinistra abbiamo chiaramente l' j -esima componente della trasformata di Fourier del vettore $\vec{a}$, a destra invece abbiamo qualcosa di poco chiaro al momento, usando l'identità

$$\omega_n^{2j} = e^{i\frac{2\pi}{n}2j} = e^{i\frac{2\pi j}{n/2}} = \omega_{n/2}^j$$

(il cosiddetto lemma di dimezzamento), possiamo scrivere

$$p(\omega_n^j) = p^{[0]}(\omega_{n/2}^j) + \omega_n^j p^{[1]}(\omega_{n/2}^j)$$

e adesso a destra riconosciamo chiaramente le trasformate di Fourier dei vettori $\vec{a}^{[0]}$ e $\vec{a}^{[1]}$. Dunque abbiamo

$$DFT_n(\vec{a}) = \begin{pmatrix} DFT_{n/2}(\vec{a}^{[0]}) \\ DFT_{n/2}(\vec{a}^{[1]}) \end{pmatrix} + \begin{pmatrix} \omega_n^0 & 0 & 0 & \dots & 0 \\ 0 & \omega_n^1 & 0 & \dots & 0 \\ 0 & 0 & \omega_n^2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \omega_n^{n-1} \end{pmatrix} \begin{pmatrix} DFT_{n/2}(\vec{a}^{[1]}) \\ DFT_{n/2}(\vec{a}^{[0]}) \end{pmatrix}$$

che d'altra parte possiamo riscrivere utilizzando il fatto che

$$\omega_n^{j+\frac{n}{2}} = e^{2\pi i \frac{j+\frac{n}{2}}{n}} = e^{\frac{2\pi i j}{n} + \pi i} = e^{\pi i} e^{\frac{2\pi i j}{n}} = -\omega_n^j$$

e dunque introducendo la matrice

$$D = \begin{pmatrix} \omega_n^0 & 0 & \dots & 0 \\ 0 & \omega_n^1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \omega_n^{\frac{n}{2}-1} \end{pmatrix}$$

abbiamo finalmente che la proprietà di sottostruttura è

$$DFT_n(\vec{a}) = \left(\frac{DFT_{n/2}(\vec{a}^{[0]})}{DFT_{n/2}(\vec{a}^{[0]})} \right) + \left(\frac{D}{0 \mid -D} \right) \left(\frac{DFT_{n/2}(\vec{a}^{[1]})}{DFT_{n/2}(\vec{a}^{[1]})} \right).$$

Dunque per calcolare ricorsivamente la trasformata abbiamo immediatamente il seguente algoritmo:

Algoritmo 2.34 (Fast Fourier Transform) — Definiamo il seguente algoritmo con input $\vec{a}$ un arbitrario vettore in $\mathbb{C}^{2^k}$:

```

1: function FFT( $\vec{a}$ )
2:    $n \leftarrow \vec{a}.len$ 
3:   if  $n = 1$  then
4:     return  $\vec{a}$ 
5:   end if
6:   Initialize empty vector  $\vec{y}$  of length  $n$ .
7:    $\vec{a}^{[0]} \leftarrow (a_0, a_2, \dots, a_{n-2})$ 
8:    $\vec{a}^{[1]} \leftarrow (a_1, a_3, \dots, a_{n-1})$ 
9:    $\vec{y}^{[0]} \leftarrow \text{FFT}(\vec{a}^{[0]})$ 
10:   $\vec{y}^{[1]} \leftarrow \text{FFT}(\vec{a}^{[1]})$ 
11:   $\omega \leftarrow e^{i\frac{2\pi}{n}}$ 
12:   $D \leftarrow 1$ 
13:  for  $i \leftarrow 0$  to  $\frac{n}{2} - 1$  do
14:     $t \leftarrow D \cdot y_i^{[1]}$ 
15:     $y_i \leftarrow y_i^{[0]} + t$ 
16:     $y_{i+\frac{n}{2}} \leftarrow y_i^{[0]} - t$ 
17:     $D \leftarrow D \cdot \omega$ 
18:  end for
19:  return  $\vec{y}$ 
20: end function

```

Analizzando la complessità di questo algoritmo troviamo che la ricorrenza è

$$T(n) = 2T(n/2) + \frac{n}{2}4$$

che per il master theorem rende immediatamente una complessità di $\Theta(n \log n)$. L'algoritmo di trasformata inversa veloce (INV_FFT) è completamente analogo per la proprietà di conformità della DFT, e si calcola in $\Theta(n \log n + n) = \Theta(n \log n)$ (dato che tutte le proprietà usate finora valgono anche per $\bar{\omega}_n$).

Utilizzando questa primitiva quindi possiamo finalmente calcolare le convoluzioni lineari in tempo efficiente, in particolare in $\Theta(n \log n)$:

Algoritmo 2.35 (Convoluzione lineare) — Dati due vettori $\vec{a}, \vec{b}$ di stessa lunghezza in un generico spazio $\mathbb{C}^n$ definiamo il seguente algoritmo:

```

1: function LINCONV( $\vec{a}, \vec{b}$ )
2:    $n \leftarrow \vec{a}.len$ 
3:   Initialize empty vectors  $X, Y$  of length  $2^{\lceil \log_2 n \rceil}$ 
4:   Initialize empty vector  $Z$  of length  $2^{\lceil \log_2 n \rceil} + 1$ 
5:    $\vec{X} \leftarrow \text{FFT}(\vec{a} \mid 0)$ 
6:    $\vec{Y} \leftarrow \text{FFT}(\vec{b} \mid 0)$ 

```

```

7:   for  $i \leftarrow 0$  to  $2n - 1$  do
8:        $Z_i \leftarrow X_i \cdot Y_i$ 
9:   end for
10:  return INV_FFT( $\vec{Z}$ )
11: end function

```

§2.5.5 Esempi notevoli

Le considerazioni fatte finora si semplificano nel caso di trasformate notevoli.

Esempio 2.36 (Vettore costante)

Calcolare efficientemente la DFT del vettore costante

$$\vec{a} = (k, k, \dots, k).$$

Soluzione. Applicando brutalmente la formula

$$DFT_n(\vec{a})_j = \sum_{m=0}^{n-1} k \omega_n^{jm}$$

che per $j \neq 0$ è

$$\dots = k \sum_{m=0}^{n-1} (\omega_n^j)^m = k \frac{(\omega_n^j)^n - 1}{\omega_n^j - 1} = 0$$

mentre per $j = 0$ abbiamo chiaramente nk , quindi

$$DFT_n(\vec{a}) = \begin{pmatrix} nk \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

Che si calcola in $\Theta(1)$. □

Esempio 2.37 (Vettore singolare)

Calcolare efficientemente la DFT del vettore

$$\vec{b} = \begin{pmatrix} b \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

Soluzione. Banalmente scrivendo la trasformata come la matrice di Fourier abbiamo

$$F_n \vec{b} = b F_n \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

che è chiaramente la prima colonna della matrice di Fourier moltiplicata per b , e quindi abbiamo

$$F_n \vec{b} = \begin{pmatrix} b \\ b \\ \vdots \\ b \end{pmatrix}.$$

Che di nuovo si calcola in $\Theta(1)$. □

Questi esempi ci fanno chiedere come si comportino in generale le applicazioni ripetute di DFT. Abbiamo quindi il seguente lemma:

Lemma 2.38 (Applicazioni ripetute della DFT)

Per ogni vettore $\vec{x} \in \mathbb{C}^n$ abbiamo che

$$DFT_n^2(\vec{x}) = n\vec{x}^R$$

dove $\vec{x}^R$ denota il *reverse* del vettore $\vec{x}$, cioè il vettore

$$\vec{x}^R = \begin{pmatrix} x_{n-1} \\ x_{n-2} \\ \vdots \\ x_0 \end{pmatrix}$$

analogamente abbiamo

$$DFT_n^3(\vec{x}) = n [DFT_n(\vec{x})]^R$$

$$DFT_n^4(\vec{x}) = n^2 \vec{x}.$$

Dimostrazione. Tutte le formule discendono dalla prima, che è brutalmente un conto. Infatti notiamo che essendo F_n simmetrica, elevarla al quadrato ci dà nell'entrata i, j il prodotto scalare tra la colonna i e la colonna j :

$$[F_n^2]_{ij} = \sum_{k=0}^{n-1} \omega_n^{ik} \omega_n^{kj} = \sum_{k=0}^{n-1} (\omega_n^{i+j})^k = \frac{(\omega_n^{i+j})^n - 1}{\omega_n^{i+j} - 1}$$

che per $i+j \not\equiv 0 \pmod{n}$ è 0, e per $i+j = n$ è esattamente n . Dunque la matrice che otteniamo è

$$F_n^2 = \begin{pmatrix} 0 & \dots & 0 & n \\ 0 & \dots & n & 0 \\ \vdots & \ddots & \vdots & \vdots \\ n & \dots & 0 & 0 \end{pmatrix} = nR_n$$

dove R_n è la matrice che manda $\vec{x}$ nel suo reverse. Dunque abbiamo la tesi. □

Da questo deduciamo anche che

$$\frac{DFT_n^3(\vec{x})}{n^2} = DFT_n^{-1}(\vec{x})$$

Esempio 2.39 (Vettori n, k -sparsi)

Siano n, k due potenze di 2. Calcolare la DFT di un vettore $\vec{x}$ tale che sia n, k -sparso. Un vettore n, k -sparso è un vettore in $\mathbb{C}^n$ tale che

$$i \not\equiv 0 \pmod{k} \implies x_i = 0$$

cioè un vettore che è 0 in tutte le componenti di indice non multiplo di k .

Soluzione. L'osservazione chiave consiste nel provare a decomporre $\vec{x}$ in componenti di indice pari e componenti di indice dispari, trovando che $\vec{x}^{[0]}$ è un vettore $\left(\frac{n}{2}, \frac{k}{2}\right)$ -sparso, e $\vec{x}^{[1]}$ è il vettore identicamente nullo (di cui evidentemente la trasformata è di nuovo il vettore identicamente nullo)! Dunque calcolare la FFT di $\vec{x}$ è molto facile, e ci basta applicare

$$DFT_n(\vec{x}) = \begin{pmatrix} DFT_{n/2}(\vec{x}^{[0]}) \\ DFT_{n/2}(\vec{x}^{[1]}) \end{pmatrix}$$

che genera immediatamente un algoritmo divide and conquer con complessità $T(n, k)$ tale che, se $k \neq n$

$$T(n, k) = \begin{cases} \Theta(n \log n) & \text{se } k = 1 \text{ (il vettore non è sparso)} \\ T\left(\frac{n}{2}, \frac{k}{2}\right) & \text{altrimenti.} \end{cases}$$

che risolvendola unfolding per induzione diventa semplicemente

$$T(n, k) = T\left(\frac{n}{k}, 1\right) = \Theta\left(\frac{n}{k} \log \frac{n}{k}\right)$$

se d'altra parte $n = k$ abbiamo che il vettore è singolare, e quindi possiamo calcolarne la trasformata in tempo costante. $\square$

§3 Dynamic Programming

Abbiamo già detto che il grosso problema del divide and conquer era la memorylessness. Questo è particolarmente lampante nel problema dei numeri di fibonacci:

Esempio 3.1 (Numeri di Fibonacci)

Dato n in input, calcolare l' n -esimo numero di Fibonacci, definito come

$$F_n = \begin{cases} 1 & \text{se } 0 \leq n \leq 1 \\ F_{n-1} + F_{n-2} & \text{altrimenti.} \end{cases}$$

Questo problema risolto in mood Naïve con un divide and conquer va in tempo $\Theta(2^n)$, decisamente non ottimale. L'algoritmo DP prevede di adoperare la cosiddetta *memoizzazione*, ovvero creiamo una "lookup table" in cui ci segniamo gli output delle sottoistanze che risolviamo durante le chiamate ricorsive in modo da non doverle ricalcolare se la sottoistanza si dovesse ripresentare.

In un certo senso quello che nel divide and conquer era l'**albero delle sottoistanze qui diventa un DAG**, in cui il modo più naturale per arrivare alla cima è iterare a partire dai livelli più bassi, fino ad arrivare alla cima. Questo è il cosiddetto approccio bottom-up. Per risolvere i problemi utilizzando un approccio top-down, come per il D&C, abbiamo bisogno della memoizzazione.

§3.1 Memoizzazione

La memoizzazione è un processo attraverso la quale possiamo implementare un algoritmo dp senza perdere la comodità implementazionale della ricorsione. Essenzialmente si basa sul salvare una certa struttura dati dinamica X tale che all'inizio viene inizializzata con una procedura chiamata `INIT_X()` e durante la ricorsione viene chiamata sulle istanze con una procedura chiamata `REC_X(I)`, dove I è un'istanza. Essenzialmente `INIT_X()` fa due cose:

1. Risolve ogni istanza di base, scrivendo in X la soluzione.
2. Per ogni istanza non di base inserisce in X un valore di default, che renda immediatamente riconoscibile il fatto che l'istanza non è ancora stata risolta.

Poi l'algoritmo `REC_X(I)` molto semplicemente fa due cose

1. Controlla se l'istanza I è stata risolta, ovvero se $X(I)$ **non** è un valore di default, in caso affermativo restituisce $X(I)$.
2. In caso negativo applica l'"algoritmo divide and conquer" ovvero applica la fase di divide, di ricorsione (su `REC_X`) e di conquer.

Notiamo che questo sposta la nostra attenzione quasi totalmente sulla struttura dati X : se costruiamo una struttura dati che implementi naturalmente la proprietà di sottostruttura del problema, allora la complessità dell'algoritmo dipenderà semplicemente da quanto velocemente riusciamo a riempire X , e la correttezza seguirà immediatamente. Un esempio tipico è fibonacci, per cui la struttura dati che ci serve è semplicemente un array 1-dimensionale che chiamiamo `fib[]`, la proprietà di sottostruttura segue direttamente dalla definizione:

$$\text{fib}[n] = \text{fib}[n-1] + \text{fib}[n-2]$$

e grazie al fatto che con questa proprietà possiamo riempire in tempo lineare l'array (semplicemente iterando a indici crescenti) l'algoritmo avrà complessità lineare $\Theta(n)$. Notiamo che ovviamente, dato un codice memoizzato non possiamo immediatamente scrivere una ricorsione come per gli algoritmi divide and conquer, perchè non sappiamo a priori quando un'istanza è stata già risolta.

Il succo del discorso in realtà qui è che **scrivere algoritmi memoizzati ha senso soltanto quando non sappiamo a priori come è fatto il DAG della sottostruttura**, cioè quando non sappiamo a priori (o possiamo saperlo, ma ci richiederebbe tanto lavoro, e quindi non abbiamo voglia di farlo/non è efficiente farlo) quali sottoistanze sono richieste per risolvere una data istanza. In una situazione del genere con un approccio bottom-up rischieremmo di risolvere più sottoistanze di quante siano effettivamente necessarie, e quindi **con un approccio top-down memoizzato risparmieremmo tempo, risolvendo solo strettamente quanto è necessario**.

§3.2 Paradigma generale DP

L'approccio DP quindi in generale per risolvere un problema computazionale è costituito dai seguenti passi:

Sottostruttura Identificazione della proprietà di sottostruttura ottima, che ci dà una funzione S che a istanze $s_1^*, s_2^*, \dots, s_k^*$ associa l'istanza

$$s^* = S(s_1^*, s_2^*, \dots, s_k^*).$$

Costo delle istanze ottime Identificazione della ricorrenza dei costi, che ci permette di ottenere una funzione f che ci dà una ricorsione (numerica) sui costi computazionali delle sottoistanze

$$c(s^*) = f(c(s_1^*), c(s_2^*), \dots, c(s_k^*))$$

in questo passo si cerca di identificare quali sono le **informazioni strutturali**, che ci interessano salvare nella struttura dati X per poter eseguire questa ricorrenza (chiaramente non vogliamo dover salvare l'intera istanza, perchè questo può risultare in costi di memoria mostruosi).

Implementazione dell'algoritmo Top-Down o Bottom-Up, in base a cosa conviene.

§3.3 Problemi su stringhe e LCS

Cominciamo dando un po' di definizioni:

Definizione 3.2 (Stringa). Dato un alfabeto Σ una *stringa* di caratteri in Σ è un elemento della sua stella di Kleene (dunque può anche essere la stringa vuota).

Definizione 3.3 (Prefissi e suffissi). Data una stringa $X \in \Sigma^*$ scritta esplicitamente come

$$X = \langle x_1, x_2, \dots, x_m \rangle$$

definiamo i suoi *prefissi* e i suoi *suffissi* come:

$$X_i := \langle x_1, x_2, \dots, x_i \rangle$$

$$X^i := \langle x_i, x_{i+1}, \dots, x_m \rangle$$

per ogni $0 \leq i \leq m + 1$. Per $i = 0$ abbiamo il *prefisso degenero*, che è semplicemente vuoto, e per $i = m + 1$ abbiamo il *suffisso degenero*, che è di nuovo vuoto. Denotiamo la stringa vuota con ε .

Definizione 3.4 (Sottostringhe e sottosequenze). Data una stringa $X \in \Sigma^*$ scritta esplicitamente come

$$X = \langle x_1, x_2, \dots, x_m \rangle$$

definiamo una sua *sottostringa* come

$$X_{i,j} := \langle x_i, x_{i+1}, \dots, x_{j-1}, x_j \rangle$$

Per ogni $0 \leq i, j \leq m + 1$. Come per le sommatorie, se i due indici sono $j < i$ assumiamo che la sottostringa $X_{i,j}$ sia vuota. Al contrario definiamo una *sottosequenza* di una stringa $X \in \Sigma^*$ come una stringa $Z \in \Sigma^*$ tale che esiste una successione di indici a_i crescente tale che

$$x_{a_i} = z_i$$

per ogni $z_i \in Z$. Cioè è un sottoinsieme di X che mantiene l'ordine di X .

Esempio 3.5 (Sottostringhe contro sottosequenze)

Notiamo che le sottostringhe di una certa stringa sono semplicemente determinate dalla scelta degli estremi, che quindi ci dà

$$\# \text{sottostringhe di una stringa di lunghezza } m = \frac{m(m-1)}{2}$$

mentre l'insieme delle sottosequenze è semplicemente l'insieme delle parti di X , cioè

$$\# \text{sottosequenze di una stringa di lunghezza } m = 2^m.$$

Il problema quindi su cui ci concentriamo d'ora in poi è il seguente:

Problema 3.6 (LCS). Date in input due stringhe $X, Y \in \Sigma^*$ dare in output una stringa $Z \in \Sigma^*$ di lunghezza massima tale che sia una sottosequenza sia di X che di Y .

In un certo senso calcolare la LCS tra due stringhe ci dà una nozione di "metrica" tra stringhe (insomma definisce una distanza). É chiaro che l'approccio Naïve non funziona per questo problema, dato che le sottosequenze sono troppe.

§3.3.1 Sottostruttura

Facciamo alcune considerazioni informali che ci porteranno alla proprietà di sottostruttura:

- Chiaramente se una delle due stringhe è vuota, la LCS sarà vuota.
- Se abbiamo $\text{LCS}(X, Y) = M$ abbiamo anche che allungando entrambe le stringhe di uno stesso carattere, abbiamo

$$\text{LCS}(\langle X, A \rangle, \langle Y, A \rangle) = \langle M, A \rangle$$

cioè la LCS aumenta di conseguenza. Ma questo vale anche al contrario! Cioè se abbiamo

$$\text{LCS}(\langle X, A \rangle, \langle Y, A \rangle) = M'$$

esisterà un M tale che

$$\text{LCS}(X, Y) = M$$

$$\text{LCS}(\langle X, A \rangle, \langle Y, A \rangle) = \langle M, A \rangle$$

e la dimostrazione è per assurdo nel modo chiaro.

- Cosa succede se abbiamo due stringhe X, Y tali che

$$X = \langle X', a \rangle$$

$$Y = \langle Y', b \rangle$$

con due terminazioni diverse? Beh chiaramente la loro LCS non terminerà **con entrambe**, cioè ci sarà una delle due lettere che è inutile al conseguimento della LCS, cioè

$$\text{LCS}(X, Y) = \langle Z, a \rangle$$

oppure, esclusivamente

$$\text{LCS}(X, Y) = \langle Z, b \rangle.$$

Questo vuol dire che sicuramente la LCS soddisfa una delle due seguenti

$$\text{LCS}(X, Y) = \text{LCS}(X_{n-1}, Y)$$

$$\text{LCS}(X, Y) = \text{LCS}(X, Y_{n-1}).$$

Abbiamo capito dove stiamo andando. Sembra tanto che in questo caso la DP si debba svolgere su **una matrice** di dimensioni $\Theta(nm)$ con cui iteriamo sugli indici delle due stringhe. Enunciamo quindi

Teorema 3.7 (Sottostruttura per la LCS)

Siano X_i, Y_j prefissi delle stringhe X, Y di lunghezze n ed m . Allora se

$$Z = \text{LCS}(X_i, Y_j) = \langle z_1, z_2, \dots, z_k \rangle$$

abbiamo che:

1. Se uno dei due prefissi è vuoto, abbiamo la stringa nulla, ovvero se $i = 0$ oppure $j = 0$ abbiamo $Z = \varepsilon$.
2. Se sia $i > 0$ che $j > 0$ abbiamo che
 - a) Se $x_i = y_j$ allora $z_k = x_i = y_j$ e quindi abbiamo

$$\text{LCS}(X_{i-1}, Y_{j-1}) = Z_{k-1}.$$

- b) Se $x_i \neq y_j$ allora Z è la stringa di lunghezza maggiore tra

$$\text{LCS}(X_{i-1}, Y_j)$$

$$\text{LCS}(X_i, Y_{j-1}).$$

Dimostrazione. Il primo caso è ovvio. Il caso interessante è il secondo:

- 2a) Se per assurdo $z_k \neq x_i$ avremmo che la stringa

$$\langle Z, x_i \rangle$$

è una sottosequenza comune di X, Y . Infatti dato che Z è una sottosequenza esisteranno successioni a_i e b_i crescenti tali che

$$x_i = z_{a_i}$$

$$y_i = z_{b_i}$$

ma d'altra parte $a_k \neq n$ e $b_k \neq m$ (dato che $y_i \neq z_k \neq x_i$) e quindi la sottosequenza formata allungando le due successioni con i termini

$$a_{k+1} = n$$

$$b_{k+1} = m$$

è effettivamente una sottosequenza comune di entrambe le stringhe, ed è più grande di Z , il che è assurdo.

- 2b) Analogamente.

E quindi abbiamo dimostrato la tesi. □

Dalla proprietà di sottostruttura scaturisce subito la ricorrenza sui costi. Se $l(i, j)$ è la lunghezza della LCS per i prefissi i e j abbiamo

$$l(i, j) = \begin{cases} 0 & \text{se una delle due stringhe è vuota, cioè } i = 0 \vee j = 0 \\ 1 + l(i-1, j-1) & \text{se entrambe le stringhe non sono vuote e } x_i = y_j \\ \max \{l(i-1, j), l(i, j-1)\} & \text{se entrambe le stringhe sono non vuote e } x_i \neq y_j. \end{cases}$$

e da questa scaturisce immediatamente l'implementazione, considerando il fatto che la struttura dati che ci memoizza è naturalmente una matrice rettangolare. L'**informazione aggiuntiva** che dobbiamo memorizzare in ogni casella della matrice è semplicemente la lunghezza della LCS associata a quell'istanza, e, per ricostruire la LCS alla fine il cosiddetto **grafo delle dipendenze** della DP, ovvero il sottografo (è un albero!) del DAG delle sottoistanze che ci permette di passare da soluzione ottima a soluzione ottima sulla base di quale caso della proprietà di sottostruttura è verificata.

IN REALTÀ non ci serve tutta questa informazione, ci basta qualcosa di lineare $\Theta(n + m)$ che ci salvi tutto il percorso a partire da $dp[n][m]$ fino a 0.

Algoritmo 3.8 (LCS) — Il codice della LCS segue immediatamente dalla discussione sopra:

Algoritmo 3.9 (LCS - Memoizzato) — contenuto...

§3.4 Matrix Chain Multiplication

Concentriamoci su un nuovo problema:

Problema 3.10 (Matrix Chain Multiplication). Dato un array di n matrici *compatibili tra loro*, ovvero che possono essere moltiplicate tra di loro nell'ordine dell'array, determinare la parentesizzazione del prodotto che minimizza il costo computazionale. Ci interessiamo di matrici a dimensioni fissate ma entrate variabili, quindi in input supponiamo di avere semplicemente un vettore $\vec{P}$ lungo $n + 1$ delle dimensioni

Facciamo alcuni commenti non banali sui costi:

- Il costo computazionale usando l'algoritmo Naïve del prodotto di due matrici $p \times q$ e $q \times r$ è

$$T(p, q, r) = pqr.$$

- Il costo computazionale **dipende** dalla parentesizzazione, pensiamo ad esempio a matrici $A_1 \in \mathbb{C}_{10 \times 100}$, $A_2 \in \mathbb{C}_{100 \times 5}$, $A_3 \in \mathbb{C}_{5 \times 50}$, i loro prodotti hanno un costo computazionale di:

$$(A_1 A_2) A_3 = (10 \cdot 100 \cdot 5) + (10 \cdot 5 \cdot 50) = 7500$$

$$A_1 (A_2 A_3) = (100 \cdot 5 \cdot 50) + (10 \cdot 100 \cdot 50) = 75000.$$

- Come prima iterare su tutte le parentesizzazioni non è fattibile in quanto sono troppe. È noto che il numero di parentesizzazioni di un'espressione di n caratteri è esattamente uguale all' n -esimo numero di Catalan $C_n = \frac{1}{n+1} \binom{2n}{n}$, che cresce esponenzialmente.
- Lo spazio delle sottoistanze

Notiamo che riconducendoci a trattare soltanto l'array delle dimensioni ci siamo ricondotti a un problema puramente numerico, in cui le matrici non c'entrano più nulla. La proprietà di sottostruttura a questo punto o possiamo intuire è la seguente:

Teorema 3.11 (Proprietà di sottostruttura per la MCM)

Supponiamo di avere una parentesizzazione ottima $T_{i,j}^*$ di una sottoistanza $T_{i,j}$ della nostra stringa di matrici. Esiste un k tale che la soluzione ottima si ottiene come concatenazione di

$$T_{i,j}^* = T_{i,k}^* + T_{k+1,j}^*$$

e le due T^* sono le soluzioni ottime alle sottoistanze $T_{i,k}$ e $T_{k+1,j}$.

Dimostrazione. È un banale cut and paste. Chiaramente possiamo decomporre ogni soluzione in due sottosoluzioni nel modo richiesto dalla tesi, per la natura delle parentesizzazioni. Inoltre se per assurdo una delle due non fosse ottimale, dato che il costo di $T_{i,j}^*$ è

$$c(T_{i,j}^*) = c(T_{i,k}^*) + c(T_{k+1,j}^*) + p_{i-1}q_kr_j$$

se per assurdo una delle due soluzioni alle sottoistanze non fosse ottimale potremmo sostituirla ad una soluzione ottimale della stessa, minorando il costo ottimale di $T_{i,j}^*$ e ottenendo un assurdo. $\square$

L'intuizione per questa proprietà deriva, come per i numeri di Catalan, dal considerare ogni parentesizzazione come identificante un albero binario completo a n foglie.

Una volta ottenuta la proprietà di sottostruttura l'algoritmo dp scaturisce immediatamente dal considerare la struttura dati naturale in cui vivono le sottoistanze del problema. In questo caso abbiamo bisogno solo di due numeri per specificare una sottoistanza (gli estremi della sottostringa), e quindi le sottoistanze vivono su una matrice $n \times n$, in particolare sulla parte della tabella con $i < j$.

Per generare una sottoistanza dalle precedenti dobbiamo iterare su tutte le sottoistanze

$$T_{i,j} = T_{i,k} + T_{k,j}$$

al variare di k tra i e j . Facendo il grafo delle dipendenze notiamo che una scansione funzionante è per esempio quella che itera sulla lunghezza della sottoistanza, ovvero sulla quantità $l := j - i + 1$, questo vuol dire iterare sulle diagonali della matrice, a partire da quella principale. Il codice dunque segue:

Algoritmo 3.12 (MCM - Iterativo) — Il codice iterativo del MCM segue:

```

1: function MCM( $\vec{P}$ )
2:    $n \leftarrow \text{length}(P) - 1$ 
3:    $m[i, j]$  empty  $n \times n$  matrix
4:    $s[i, j]$  empty  $n \times n$  matrix
5:   for  $i \leftarrow 1$  to  $n$  do ▷ Casi Base
6:      $m[i, i] \leftarrow 0$ 
7:   end for
8:   for  $l \leftarrow 2$  to  $n$  do ▷ Scansione per diagonali
9:     for  $j \leftarrow l$  to  $n$  do
10:       $i \leftarrow j - l + 1$ 
11:       $m[i, j] \leftarrow \infty$ 
12:      for  $k \leftarrow i$  to  $j - 1$  do
13:         $q \leftarrow m[i, k] + m[k + 1, j] + p[i - 1]p[j]p[k]$ 
14:        if  $q < m[i, j]$  then
15:           $m[i, j] \leftarrow q$ 

```

```
16:          $s[i, j] \leftarrow k$ 
17:     end if
18: end for
19: end for
20: end for
21: return  $m[1, n]$ 
22: end function
```

§4 Greedy

Il paradigma greedy nasce dalla realizzazione che tutte le DP sono solo un insieme di "scelte": ogni istanza viene risolta "scegliendo" solo alcune delle sottoistanze del DAG e combinandone le soluzioni per arrivare ad una soluzione dell'istanza originale.

Un approccio greedy riduce di molto il costo computazionale di un algoritmo, perché itera dall'alto verso il basso sull'albero delle sottoistanze, risolvendo **un'unica** sottoistanza per livello. La procedura è quindi:

Scelta incauta: per l'istanza corrente viene identificata una sottoistanza che permetta di risolvere "da sola" l'istanza originale. Questa scelta avviene attraverso "proprietà locali".

Sottoistanza residua: Si "ripulisce" l'istanza originale dagli elementi non compatibili con la scelta effettuata, ottenendo un'unica sottoistanza. Si **ricorre** sull'algoritmo, cercando di risolvere la sottoistanza residua in una cosiddetta "tail-recursion" (cioè l'ultima cosa che facciamo è chiamare la ricorsione, e poi fare un conquer a costo 0).

Notiamo che il motivo per cui riusciamo a risparmiare così tanta complessità computazionale è che stiamo assumendo qualcosa di più forte sulla struttura delle sottoistanze del problema, ovvero stiamo assumendo che

Proprietà di sottostruttura: Ci sia, come è successo finora, una proprietà di sottostruttura di "dipendenza tra sottoistanze" (ovvero, ogni sottoistanza può essere risolta avendo sufficienti sue sottoistanze).

Proprietà di scelta greedy: Ci sia una cosiddetta **proprietà di scelta greedy**, che permette di isolare un'unica sottoistanza per ogni istanza nel DAG delle sottoistanze.

Dunque la correttezza di un algoritmo greedy seguirà dalla verifica della correttezza di queste due proprietà.

§4.1 Task Scheduling

Un problema classico risolvibile con un approccio greedy è il seguente

Problema 4.1 (Task Scheduling). Definiamo il seguente problema computazionale:

Input: Un insieme I di n attività $a_1, a_2, \dots, a_n$, ovvero intervalli della forma $[s_i, f_i)$ (chiaramente si ha $s_i \leq f_i$ per ogni i).

Output: Un sottoinsieme massimale O di I tale che tutti gli elementi di O siano disgiunti a due a due.

Informalmente abbiamo n attività a_i , ciascuna con tempo di inizio s_i e di fine f_i e vogliamo trovare il modo di farne il massimo numero possibile senza che si accavallino.

Il primo candidato per la scelta greedy è prendere sempre l'attività che "finisce prima", perché ci dà più spazio per mettere altre attività dopo. Con questa scelta greedy anche la fase di cleanup sembra chiara. Già solo così abbiamo un algoritmo candidato:

Algoritmo 4.2 (Greedy1 ricorsivo - Task Scheduling) — Definiamo il seguente algoritmo assumendo che il vettore delle coppie (s_i, f_i) sia ordinato in modo che f_i sia monotonamente crescente.

```
1: function REC_GREEDY_SEL( $\vec{s}, \vec{f}, i$ )
2:    $n \leftarrow \vec{s}.len$ 
```

```

3:   if  $i > n$  then
4:       return  $\emptyset$ 
5:   end if
6:    $G \leftarrow \{i\}$  ▷ Scelta greedy
7:    $j \leftarrow i + 1$ 
8:   while  $s_j < f_i$  and  $j \leq n$  do ▷ Clean up
9:        $j \leftarrow j + 1$ 
10:  end while
11:  return  $G \cup \text{REC\_GREEDY\_SEL}(\vec{s}, \vec{f}, j)$ 
12: end function

```

Che è ricorsivo, ma è chiaramente implementabile naturalmente in modo iterativo:

Algoritmo 4.3 (Greedy1 iterativo - Task Scheduling) — Definiamo il seguente algoritmo assumendo che il vettore delle coppie (s_i, f_i) sia ordinato in modo che f_i sia monotonamente crescente.

```

1: function IT_GREEDY_SEL( $\vec{s}, \vec{f}$ )
2:    $n \leftarrow \vec{s}.len$ 
3:    $A \leftarrow \{1\}$ 
4:    $g \leftarrow 1$ 
5:   for  $i = 2$  to  $n$  do
6:       if  $s_i \geq f_g$  then
7:            $A \leftarrow A \cup \{i\}$ 
8:            $g \leftarrow i$ 
9:       end if
10:  end for
11: end function

```

Un cosiddetto approccio "sliding window".

Dimostrare la correttezza qui è la parte difficile, dato che la complessità è molto chiaramente quella del for, cioè $\Theta(n)$. Per dimostrare la correttezza abbiamo bisogno di dimostrare due cose:

La proprietà di scelta greedy: dobbiamo dimostrare che esiste una soluzione ottimale che include la scelta greedy: in questo caso dobbiamo dimostrare che, assumendo le attività siano ordinate per tempo di fine crescente, esiste una soluzione ottimale contenente a_1 la prima attività.

La proprietà di sottostruttura: Dobbiamo dimostrare che la soluzione alla sottoistanza residua dopo una scelta greedy e il relativo cleanup porti effettivamente a risolvere l'istanza originale.

Lemma 4.4 (Proprietà di scelta greedy)

Per ogni istanza $I = \{a_1, \dots, a_n\}$ con gli a_i ordinati per

Lemma 4.5 (Proprietà di sottostruttura)

contenuto...

§4.2 Compressione dati

Un algoritmo di compressione dati in generale è una procedura che prende in input un file di testo $F \in C^*$, dove C è l'alfabeto dei caratteri ASCII, e restituisce in output una *rappresentazione binaria compatta* di F detta *codifica*. Dunque l'output è in $\{0, 1\}^*$. Questa deve avere alcune proprietà:

Invertibilità: La codifica deve essere invertibile, ovvero la funzione deve essere iniettiva: file binari distinti devono essere mandati in codifiche distinte

Efficienza: La codifica deve essere gestita da un algoritmo efficiente sia in tempo che in memoria.

Una metrica per quantificare la bontà di una codifica è il rapporto tra la lunghezza del file in chiaro con quello codificato, per esempio la codifica ASCII ha un rapporto costante a 8 bit per carattere. Quello che studieremo è un sistema di codifica che associa *codeword* a caratteri, ovvero allo stesso carattere nel file F è associata la stessa codifica: se $e : C^* \rightarrow \{0, 1\}^*$ è la codifica abbiamo

$$e(F) = e(\langle c_1, c_2, \dots, c_{|F|} \rangle) = \langle e(c_1), e(c_2), \dots, e(c_{|F|}) \rangle$$

un modo di fare ciò può essere analizzare la frequenza dei vari caratteri nel file, ottenendo una lista f_i di frequenze tali che

$$\sum_{i=1}^{256} f_i = 1$$

e associare a caratteri a maggior frequenza codeword più corte. Questo però in modo naïve non è facile da fare, per esempio se associamo

Carattere	a	b	c	d	e	f
Frequenza	0.45	0.13	0.12	0.16	0.9	0.5
Codeword	0	10	11	1	100	101